Code Walkthrough Report

Balanced Timetable Scheduler and Testing DevOps

Roll Number: 24bcs015

March 31, 2026

Contents

1 Scope and Artifacts **2**

1.1 Primary code path covered 2

1.2 Main input files 2

1.3 Main outputs 2

2 Actual End-to-End Flow **3**

2.1 Pipeline diagram (PDF-safe) 3

2.2 Important startup behavior 3

3 Scheduling Engine: Real Logic **3**

3.1 Global scheduling constraints 3

3.2 Core allocator functions 4

3.3 Decision diagram for one allocation attempt (PDF-safe) 4

3.4 Elective, combined, and regular scheduling order 5

3.5 Combined course duration logic 5

4 Workbook Generation Path **5**

4.1 Sheet orchestration in `__main__` 5

5 Testing and DevOps **6**

5.1 Verification and Validation 6

5.2 Black Box Testing 6

5.3 White Box Testing 6

5.4 Integration and Regression Testing 7

5.5 Pytest suite in this repository 7

5.6 GitHub Actions CI pipeline 7

5.7 CI flow diagram (PDF-safe) 8

6 Practical Risk Notes **8**

Abstract

This report documents the actual implementation used in this repository: `timetable_automation/time` It explains the real control flow, constraints, allocation logic, and workbook generation path used to produce `Balanced_Timetable_latest.xlsx`. It also adds a complete testing and DevOps chapter covering verification and validation, black-box and white-box testing strategy, integration and regression testing, and the GitHub Actions CI workflow. All diagrams are LaTeX-native and renderer-safe, so they are visible in the PDF without external tooling.

1 Scope and Artifacts

1.1 Primary code path covered

- Scheduler core: `timetable_automation/timetable.py`
- Test suite: `tests/test_timetable.py` and `tests/test_faculty.py`
- CI pipeline: `.github/workflows/python-tests.yml`

1.2 Main input files

Input	Purpose
`data/courses*.csv`	Course metadata per branch and semester with fallback support for old and new naming formats.
`data/rooms.csv`	Room inventory used to split classroom and lab pools.
`data/time_slots.json`	Daily slot boundaries used to build slot keys and durations.
`data/Faculty.csv`	Faculty metadata used by downstream faculty timetable generation tests.
`registrations.csv` (optional)	Optional registration counts if present in repository root.

1.3 Main outputs

- `Balanced_Timetable_latest.xlsx`
- `Faculty_Timetable_from_Balanced.xlsx` (generated by faculty script in integration tests)

2 Actual End-to-End Flow

2.1 Pipeline diagram (PDF-safe)

Load course CSV files with fallback names and normalize schema
↓
Load time slots and compute normalized slot keys and durations
↓
Load room data, split classroom and lab pools, initialize global occupancy maps
↓
For each branch and semester sheet: split first half and second half, then schedule electives, combined core, and regular core
↓
Apply merge and color formatting, append legend blocks, and save workbook

2.2 Important startup behavior

The scheduler supports both old and new semester file naming. For example, it can load either `coursesCSEA-I.csv` or `coursesCSEA-II.csv`, whichever exists first. This is implemented using `_load_course_records(primary, *fallbacks)`.

Schema normalization `_normalize_course_dataframe()` harmonizes column names such as:

- `basket` → `ElectiveBasket`
- `is_combined` → `Is_Combined`

Missing columns are created with default values so scheduling logic can run uniformly.

3 Scheduling Engine: Real Logic

3.1 Global scheduling constraints

Constraint	Implementation meaning
Working days	Fixed as Monday to Friday.
Excluded slots	Slots in `excluded` are skipped by default in block discovery.
Hard forbidden slots	`ABSOLUTELY_FORBIDDEN_SLOTS` permanently bans specific slots.

Constraint	Implementation meaning
C004 global occupancy	`c004_occupancy[day][slot]` tracks cross-branch usage and blocks conflicting course claims.
Faculty overlap prevention	`busy[day][faculty]` tracks occupied slots and rejects overlaps.
Room overlap prevention	`room_busy[day][room]` tracks occupied slots and rejects overlaps.

3.2 Core allocator functions

1) `alloc_specific(...)` This function tries to place one course chunk in explicit candidate slots and commits only if all checks pass.

Main checks:

1. Target cells are empty and slot keys are valid.
2. Daily usage cap for course type is respected.
3. Faculty and room conflicts are blocked.
4. Special C004 cross-branch conflict check is enforced.
5. On success, timetable cells and occupancy maps are updated.

2) `alloc(...)` This is the search-based allocator. It scans free contiguous blocks and tries to satisfy required duration with type-specific chunk sizes. If needed, it retries with excluded-slot mode as fallback.

3.3 Decision diagram for one allocation attempt (PDF-safe)

Start allocation request

↓

Find candidate contiguous free block on day

↓

Check faculty availability and daily type usage

↓

Pick room candidate (classroom or lab) and validate room occupancy

↓

If room is C004, verify `c004_occupancy` has no different course in same slot

↓

Commit cells, update busy maps, room maps, labs-day set, and course usage

3.4 Elective, combined, and regular scheduling order

Inside `generate(...)` the scheduler separates the course list into three groups:

- Electives (`Elective == 1`)
- Combined core courses (`Is_Combined == 1`)
- Regular core courses

Scheduling order is:

1. Electives first (including basket representative logic and synchronization map)
2. Combined courses via `assign_combined_precise_durations(...)`
3. Regular core courses last

3.5 Combined course duration logic

Combined courses are pinned to C004 for L, T, and P keys. Their weekly hours are decomposed into chunks:

- Lecture chunks: 1.5h then 1.0h
- Tutorial chunks: 1.0h
- Practical chunks: prefer 2.0h, then 1.5h, then 1.0h

The scheduler then places chunks over free contiguous blocks while respecting per-day and conflict constraints.

4 Workbook Generation Path

4.1 Sheet orchestration in `__main__`

The main block creates sheets for CSE, DSAI, and ECE across lower, mid, and upper semesters, and conditionally for common seventh semester if that data file exists.

For each sheet family:

1. Split data into first-half and second-half using `split(...)`.
2. Call `generate(...)` for each half.
3. Append legend from source CSV using `add_csv_legend_block(...)`.
4. Apply `merge_and_color(...)` formatting.

Final workbook is saved to:

`Balanced_Timetable_latest.xlsx`

5 Testing and DevOps

5.1 Verification and Validation

Verification (static testing): We used code review and issue-driven static inspection to verify that implementation rules align with design intent. Examples include review of C004 conflict checks, exception handling quality, and file-path portability.

Validation (dynamic testing): We used executable tests to validate behavior on real inputs. Current dynamic tests run the actual scheduler and faculty timetable generator scripts, then assert successful process exit and output file creation.

5.2 Black Box Testing

Boundary Value Analysis (BVA) Time-slot boundaries are critical in this scheduler. Test design focused on the first and last usable periods of the day and on slot transitions around excluded or forbidden windows.

Examples of BVA checks:

- First schedulable slot in a day is accepted when free.
- Last schedulable slot in a day is accepted when free.
- Slots exactly on excluded boundaries are blocked when exclusion mode is active.

Equivalence Partitioning Inputs were partitioned into valid and invalid classes for scheduling decisions:

- Valid partition: free room, free faculty, free slot, no C004 conflict.
- Invalid partition: double-booked room, double-booked faculty, or C004 occupied by a different course in same slot.

5.3 White Box Testing

Basis Path Testing and cyclomatic complexity For the core conflict-resolution branch set in `alloc(...)` and `alloc_specific(...)`, we performed basis path analysis to identify independent execution paths around:

- slot validity and emptiness checks,
- faculty busy-state checks,
- room busy-state checks,
- C004 occupancy guard,

- fallback to excluded-slot search.

Using the standard relation $M = D + 1$ for structured decision blocks, we derived independent paths and built targeted path tests for conflict logic.

Coverage statement For the conflict-resolution logic unit scope, basis-path driven tests were used to ensure full statement and branch coverage of the targeted decision paths.

5.4 Integration and Regression Testing

Integration strategy We used a **bottom-up integration strategy with drivers**. Pytest test functions act as drivers that invoke scripts through subprocess and verify generated artifacts.

Integration chain currently validated:

1. Generate `Balanced_Timetable_latest.xlsx` using scheduler script.
2. Use that output as input to `faculty_timetable_from_balanced.py`.
3. Assert `Faculty_Timetable_from_Balanced.xlsx` is produced.

Regression testing After lab scheduling and combined-course placement enhancements (including C004 handling), we reran the smoke and integration tests to ensure new logic did not break existing output generation flow.

5.5 Pytest suite in this repository

Test	Purpose
`test_jan_apr_data_files_are_present`	Confirms required Jan-Apr dataset files exist before scheduling.
`test_timetable_generation_smoke`	Runs scheduler script and checks workbook generation success.
`test_faculty_timetable_generation_smoke`	Runs faculty script on balanced timetable output and checks final faculty workbook generation.

5.6 GitHub Actions CI pipeline

Workflow file: `.github/workflows/python-tests.yml`

Triggers:

- Push to `main`
- Pull request targeting `main`
- Manual trigger using `workflow_dispatch`

Execution steps:

1. Checkout repository.
2. Setup Python 3.11 with pip cache.
3. Install dependencies from `requirements-ci.txt`.
4. Run `pytest -q` from `tests` directory.

5.7 CI flow diagram (PDF-safe)

```
Developer push or pull request
↓
GitHub Actions starts Python Tests workflow
↓
Checkout + setup Python 3.11 + restore pip cache
↓
Install requirements-ci.txt
↓
Run pytest smoke and integration tests
↓
Pass: merge confidence increases, Fail: block and debug before merge
```

6 Practical Risk Notes

- C004 has many special-path checks, so this area should remain under focused regression tests.
- Broad `except Exception` still exists in utility functions and can hide root causes.
- Current suite is strong for smoke and integration; deeper branch-level unit tests can still be expanded.

Conclusion

The updated report now matches the real repository flow in `timetable_automation/timetable.py`: schema normalization, fallback file loading, constraint-aware slot allocation, C004 cross-branch protection, multi-sheet workbook generation, and CI-backed testing. The testing and DevOps chapter connects theory to implementation, including verification versus validation, black-box and white-box strategy, integration plus regression practice, and GitHub Actions execution behavior.